

 QUALITY & TEST ENGINEERING

TicketWave — Testing Strategy & Gates

An architectural overview of the testing layers, strict JaCoCo coverage gates, PostgreSQL container isolation, and automated CI pipelines.

SECTION 1

Test Layers & Execution Strategy

Multilayer verification spanning unit tests, integration suites, frontend Vitest, and Playwright E2E.

Multilayer Testing Architecture

Testing Layer	Framework & Tooling	Scope & Runner	Suite Volume
Backend Unit & MockMvc	JUnit 5 + Mockito + MockMvc	Services, mappers, filters, controllers (Surefire)	670 tests / 92 classes
Backend Integration	@SpringBootTest + Testcontainers	Repository queries & real DB flows (Failsafe)	35 tests / 8 IT classes
Frontend Unit / Spec	Vitest + Angular TestBed	Standalone components, guards, interceptors	401 tests / 57 specs
End-to-End (E2E)	Playwright + Chromium	Auth flows & search → book → pay golden paths	5 E2E tests / 2 specs
Performance & Load	k6 Load Runner	Search browsing & concurrent seat-hold contention	Automated script

| Real PostgreSQL & Testcontainers



Real Database Realism (No H2)

Strict Realism: Liquibase schema, transaction-abort on constraint violations, `FOR UPDATE` locks, and enum/JSON columns diverge in H2.

Idempotency Validation: H2 would pass tests while production database paths broke under real PostgreSQL constraints.



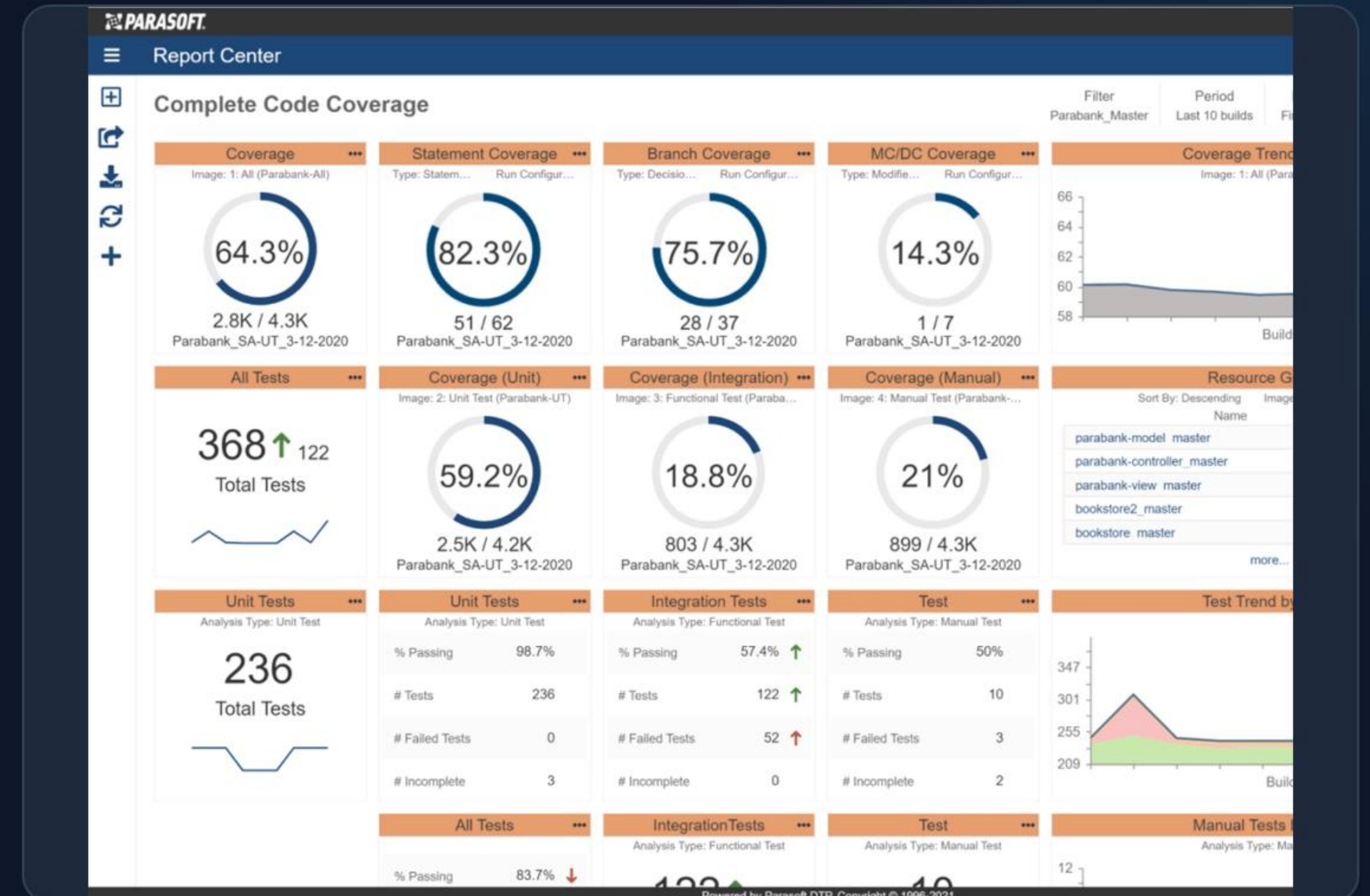
Testcontainers Isolation

Throwaway Containers: Started per JVM via `PostgresContainerSupport` and torn down by Ryuk on exit.

Zero External State: No configured database URL can resolve to real data, guaranteeing zero accidental state pollution.

Frontend & End-to-End Automation

- 🔧 **Vitest Component Specs:** 401 tests running via `npm test` (one-shot CI execution with `--watch=false`).
- 👤 **Playwright E2E Workflows:** Validates critical golden path from search to checkout and payment confirmation.
- 🛡️ **Key Test Focus:** Primary emphasis placed on seat-hold countdown precision, checkout validations, and error state rendering.



SECTION 2

Coverage Gates & Enforcement

JaCoCo automated build gates, 100% financial coverage rules, and deliberate gap documentation.

Enforced JaCoCo Coverage Gates



Core financial modules(`PricingServiceImpl` , `SeatHoldServiceImpl` , `SeatHoldExpirationScheduler` , `RefundPolicyService`)enforce strict 100% coverage gates.

| Coverage Exclusions & Gaps



Configured Exclusions

Generated Code: MapStruct implementations (`*MapperImpl.class`) and Lombok generated code (`@Lombok.Generated`) are excluded.

Dev Fixtures: `DevDataSeeder` is omitted as it never executes outside local seeding.



Deliberate Uncovered Branches

Only 9 branches remain uncovered across the bundle—all deliberate (e.g., defensive `@PrePersist` defaults or unreachable loop bounds).

Every gap is annotated with a inline note comment in the source code.

CI Pipeline & Execution Flow

GitHub Actions Architecture

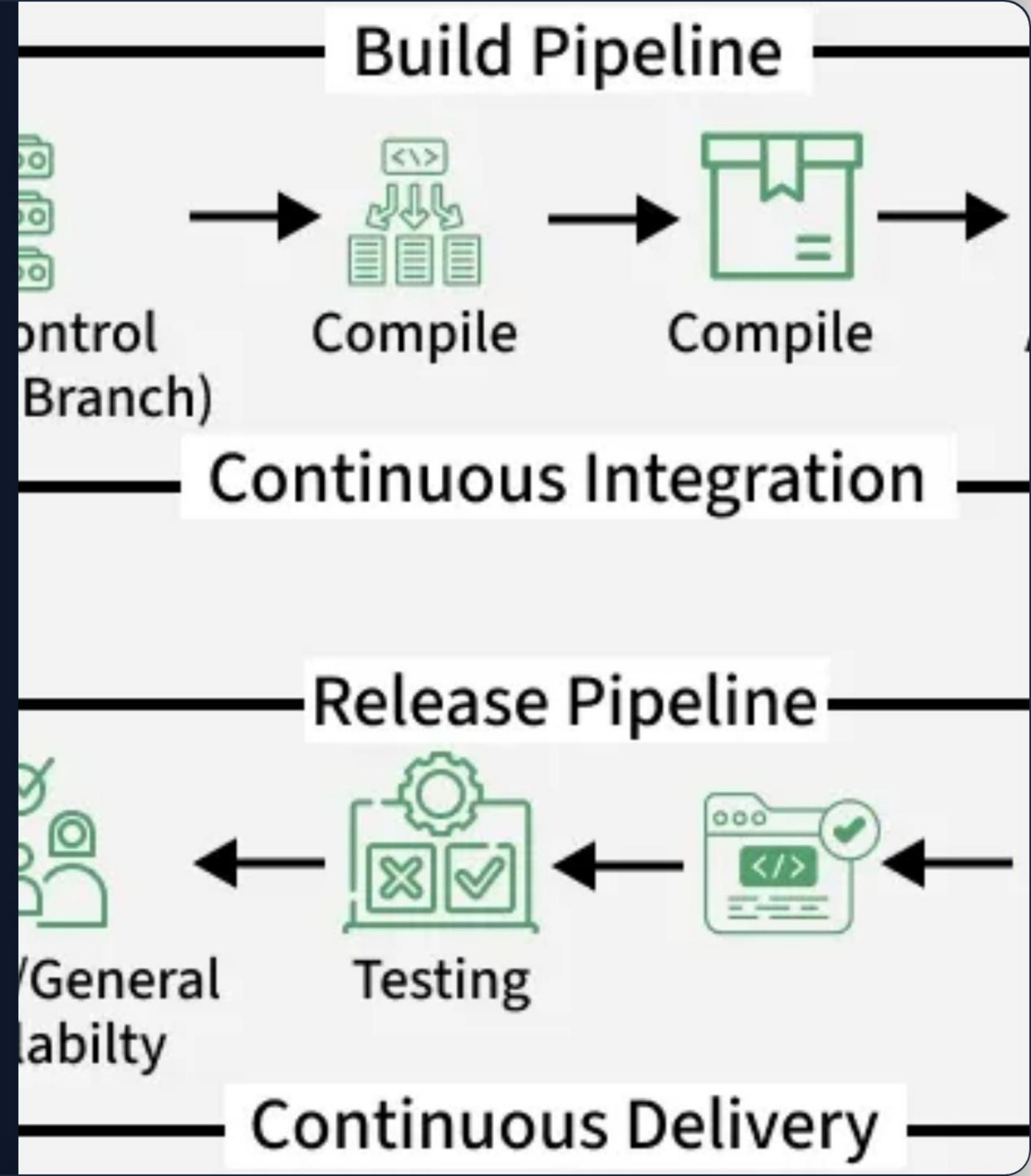
Executes three parallel jobs on push/PR: `backend` (`mvn verify` with PostgreSQL service container), `frontend` (Vitest & build), and `e2e` (Playwright).

Merging Unit & Integration Data

Executes `jacoco.exec` and `jacoco-it.exec` merging during `verify` phase, ensuring `*IT` runs contribute to authoritative gates.

Isolated E2E Environment

The E2E job operates on a third isolated container (`ticketwave_e2e`) with automated readiness probes pointing to `/v3/api-docs` .



✓ TICKETWAVE TEST SUITE VERIFIED

Questions & Discussion

670 Backend Tests | 401 Frontend Specs | 5 E2E Suites | 100% Financial Gate

```
mvn clean verify && npm test
```

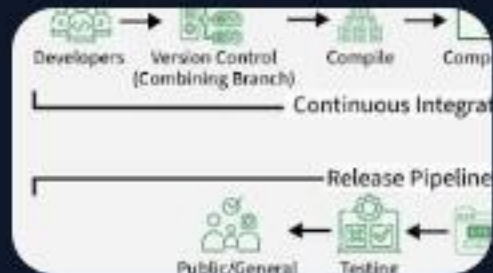

Image Sources



https://media.parasoft.com/cdn-cgi/imagedelivery/iu4N-Xe4Xq_f9Hw7qvnDtA/8750d458-d523-4aa9-3c94-93682f327100/w=1600,fit=crop,quality=85,format=auto

o

Source: www.parasoft.com



<https://media.geeksforgeeks.org/wp-content/uploads/20250821153350614262/cicd12.webp>

Source: www.geeksforgeeks.org